Practical Lab
Local AI with Ollama

Running and Developing Local AI Applications

Author	Assia Boullail
Status	PhD Student
Laboratory	LabTIC
Institution	ENSA Tanger
Date	25 October 2026

1. Introduction

Local AI means running a language model on one's own computer instead of sending requests to a cloud provider. In this lab I used **Ollama**, a tool that downloads models, loads them into memory, runs them on the CPU or GPU, and offers a simple HTTP service on *localhost:11434*. Because every request stays on the machine, prompts and documents remain private, the system works without Internet once the model is downloaded, and no per-token fee is charged.

I worked with two models, **qwen3.5** and **gemma3**. Starting from a single API call, I built a Python client, a chatbot with conversation memory, a small document-based question answering system (mini-RAG), and finally a Flask web application called *FSTT Local AI Assistant*. I also benchmarked both models on the same prompt to compare speed and memory use.

The next section shows the evidence of each step (screenshots). Then comes the benchmark, the answers to the fifteen report questions, and a conclusion.

2. Screenshots

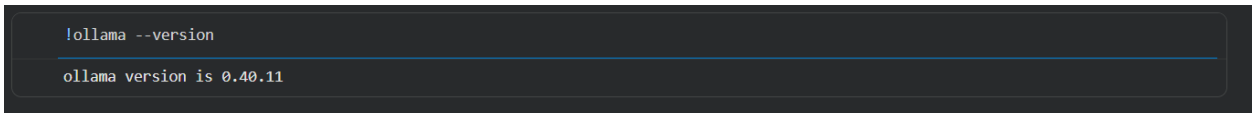


Figure 1 - Ollama installed (version 0.40.11 shown by ollama --version).



Figure 2 - Model selection and download: ollama pull and ollama list showing qwen3.5 (6.6 GB) stored on disk.

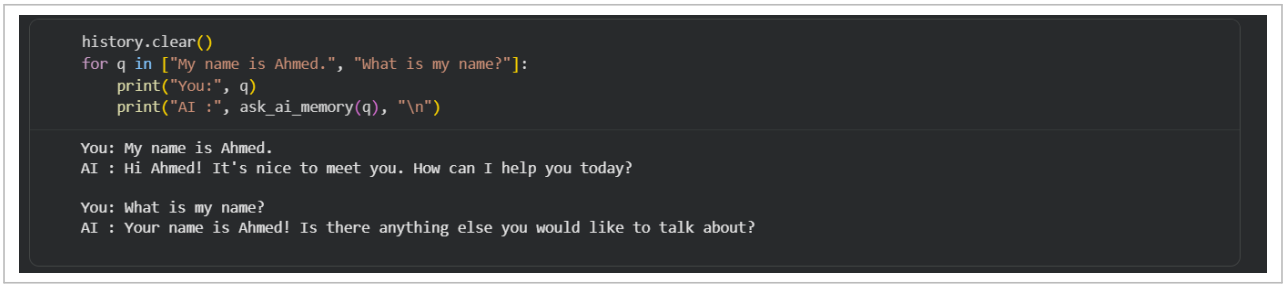


Figure 3 - Chatbot with conversation memory: after being told its name, the assistant answers 'Your name is Ahmed' because the history is re-sent with each request.

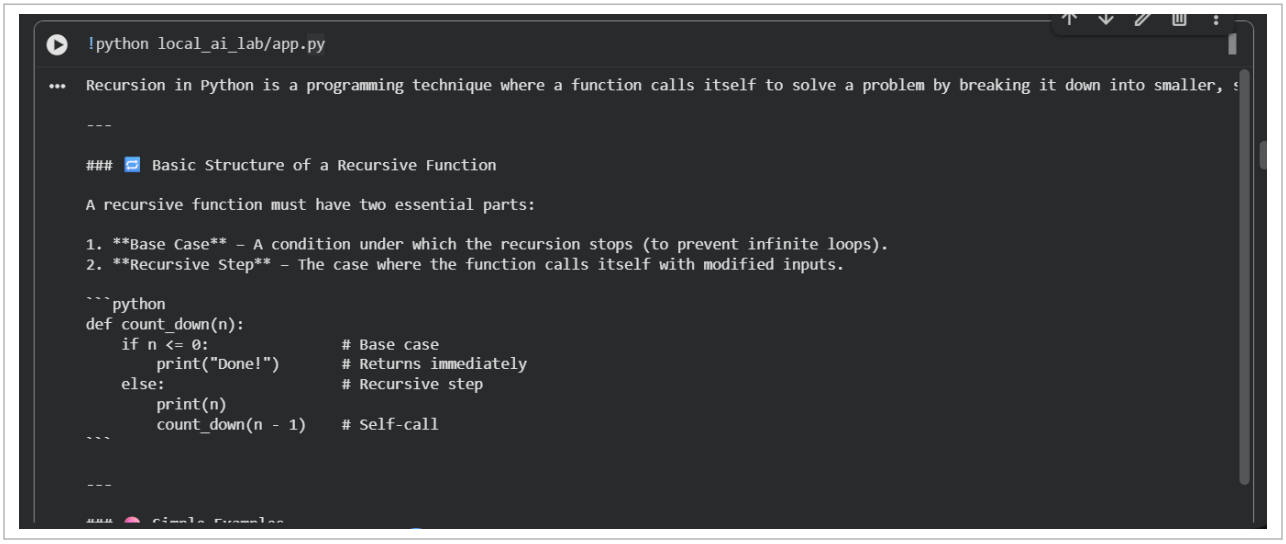


Figure 4 - Python application (app.py) sending an HTTP POST to Ollama and printing the model's explanation of recursion.



Figure 5 - Mini-RAG: for each question the retriever picks a document and the model answers from it. The off-topic question (2018 World Cup) correctly returns 'Information not available.'

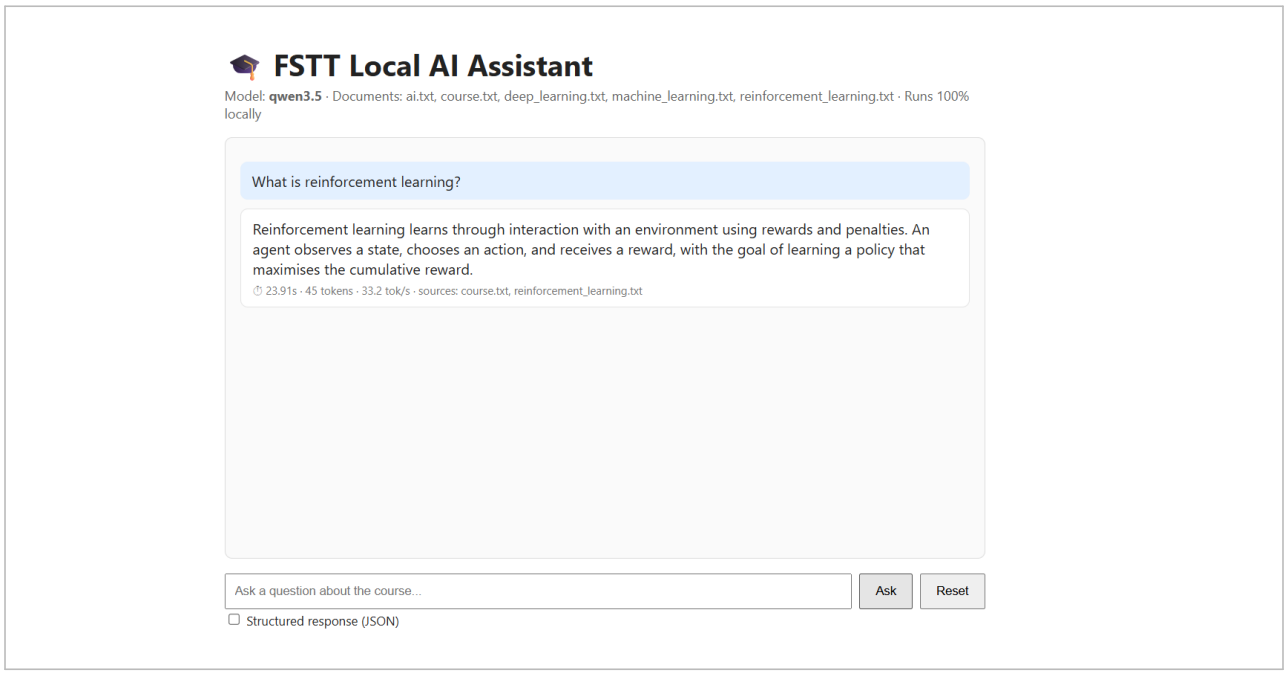


Figure 6 - The FSTT Local AI Assistant web application, showing the answer, the response time (23.91 s), tokens, speed and the source documents used.

3. Benchmark of the two models

Both models received the same prompt ('Write a 500-word explanation of reinforcement learning.') with a cold start (the model was unloaded before each run, so the load time is included in the measurements). The figures below come directly from my run.

```
import benchmark; importlib.reload(benchmark)

bench_df = pd.concat([benchmark.bench(m) for m in (MODEL, MODEL2)], ignore_index=True)
bench_df["Quality /5"] = None   # <- fill in after reading both answers
bench_df.round(2)
```

	Model	Disk size (GB)	Loaded size (GB)	VRAM (GB)	Load time (s)	Response time (s)	Tokens	Tokens/s	System RAM used (%)	Quality /5
0	qwen3.5	6.55	5.61	5.61	30.07	56.22	646	24.99	20.1	None
1	gemma3	3.35	2.88	2.88	4.68	19.27	825	57.27	23.2	None

Figure 7 - Benchmark output (bench_df) from the notebook.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 3, figsize=(13, 3.5))
bench_df.plot.bar(x="Model", y="Tokens/s", ax=ax[0], legend=False, title="Speed (tokens/s)", rot=0)
bench_df.plot.bar(x="Model", y="Loaded size (GB)", ax=ax[1], legend=False, title="Memory (GB)", rot=0)
bench_df.plot.bar(x="Model", y="Response time (s)", ax=ax[2], legend=False, title="Latency (s)", rot=0)
plt.tight_layout(); plt.show()
```



Figure 8 - Speed, memory and latency of the two models.

Model	Loaded size (GB)	Load time (s)	Response time (s)	Tokens	Tokens/s	Quality /5
qwen3.5	5.61	30.07	56.22	646	24.99	____
gemma3	2.88	4.68	19.27	825	57.27	____

Reading the results. gemma3 generated about 2.3 times more tokens per second than qwen3.5 (57.27 against 24.99) and answered in 19.27 s instead of 56.22 s, almost three times faster overall. It also loaded in 4.68 s instead of 30.07 s and occupied only 2.88 GB of memory, which is about 2.7 GB less than qwen3.5 (5.61 GB). In both cases the loaded size equals the VRAM figure, so the whole model fitted on the GPU and no layers were pushed to the slower system RAM.

4. Answers to the report questions

Q1. What is Local AI?

Local AI is the practice of running an AI model, here a large language model, on hardware that I control: a laptop, a workstation or an in-house server. The weights, the prompts and the answers never leave the machine, so the system keeps working offline and has no usage fees.

Q2. What is the difference between a Cloud LLM and a Local LLM?

A cloud LLM runs in the provider's data centres and is reached over the Internet through a paid API. It gives access to very large models without any hardware on my side, but data leaves my computer, costs recur, and I depend on the provider and on my connection. A local LLM runs on my own machine. It is private, free to use and available offline, but it is limited by my hardware, so the models are smaller or compressed, and I am responsible for installing, updating and securing it.

Q3. What role does Ollama play?

Ollama is the layer between the model and the applications. It downloads and stores models, loads them into RAM or VRAM, runs the inference on the CPU or GPU, and applies each model's chat format. It then exposes a REST API on localhost:11434 (for example /api/generate, /api/chat and /api/tags), which means the same model can be used from the terminal, from Python or from a web application.

Q4. What is model quantization?

Quantization lowers the precision used to store the weights, for instance from 16-bit numbers to 8-bit or 4-bit ones. The file becomes much smaller and generation faster, with only a small loss of accuracy. This is why a model with several billion parameters can run on an ordinary computer.

Q5. Why does model size affect memory consumption?

Most of the memory is taken by the weights, so memory is roughly the number of parameters multiplied by the bytes per weight. A model with twice as many parameters needs about twice as much memory, and extra space is needed for the context (KV cache). My measurements show the same trend: qwen3.5 was 6.55 GB on disk and used 5.61 GB when loaded, while gemma3 was 3.35 GB on disk and used 2.88 GB. If a model does not fit in VRAM, part of it moves to the CPU or to swap and the speed collapses.

Q6. Why can a smaller model sometimes be preferable?

A smaller model fits on modest hardware, loads quickly and answers quickly. For well-defined tasks such as question answering over supplied documents, extraction or short summaries, its quality is often good enough. In my benchmark gemma3 loaded in 4.68 s and ran at 57.27 tokens/s, while the larger qwen3.5 needed 30.07 s to load and ran at 24.99 tokens/s.

Q7. What are the advantages of local inference?

The main advantages are privacy (data stays on the machine), offline operation, no per-token cost, no network latency, no rate limits, and full control over the model version and its settings, which also makes results easier to reproduce. It also helps with compliance when documents are confidential.

Q8. What are the disadvantages of local inference?

Local models are usually less capable than the largest cloud models. They need enough RAM or a GPU, and they can be slow on a CPU. I must install, update and monitor everything myself, the context length may be limited, and the machine's energy use and security are my responsibility.

Q9. Explain the architecture of your Python application.

The final application is a Flask web app. The browser sends the question to the /ask route. Flask asks the retriever (rag.py, a keyword TF-IDF search over the documents folder) for the two most relevant documents, builds a message made of a system instruction, the last conversation turns, the retrieved context and the question, and sends it to Ollama's /api/chat endpoint on localhost:11434. The local model produces the answer, and Flask returns it to the browser together with the sources, the response time, the number of tokens and the speed. A checkbox can request a structured JSON answer, which uses a JSON schema. The server listens on 127.0.0.1 only, so everything stays local.

Q10. Explain the role of the REST API.

The REST API is how my program talks to the model. The program sends an HTTP POST with a JSON body (model name, prompt or messages, options) and receives a JSON answer. This separates the application from the inference engine: any language or tool that can send HTTP requests can use the same local model, and the application does not need to know how inference works internally.

Q11. What is RAG?

Retrieval-Augmented Generation first searches a collection of documents for the passages related to the question, and then gives those passages to the model as context. The model answers from that context instead of from memory alone. This brings private or up-to-date knowledge without retraining, reduces invented answers, and shows which document was used. In my test, the question about the 2018 World Cup found no relevant document, and the system answered 'Information not available.'

Q12. What is the difference between an LLM and a multimodal LLM?

A standard LLM reads and writes text only. A multimodal LLM can also take other inputs, mainly images and sometimes audio or video, through an encoder that turns them into a representation the language model can use. It can therefore describe a picture, read text inside it, or answer questions about a chart.

Q13. Compare two local models experimentally.

I ran the same prompt on qwen3.5 and gemma3 with a cold start each time. The results are in the table of Section 3. gemma3 was clearly faster (57.27 against 24.99 tokens/s), had a shorter response time (19.27 s against 56.22 s), loaded faster (4.68 s against 30.07 s) and used about half the memory (2.88 GB against 5.61 GB). gemma3 also produced a longer answer (825 tokens against 646). My quality scores out of 5 are written in the table. Since qwen3.5 is the larger model, it may give more detailed or more accurate answers on some tasks, but on this machine the cost is slower responses and higher memory use.

Q14. Which model would you deploy on a computer with limited RAM?

I would deploy gemma3. It needed only 2.88 GB when loaded, so it fits comfortably on a computer with 8 GB of RAM and leaves room for the operating system and the context. It also responds quickly and starts in under five seconds. To make up for its smaller size, I would pair it with RAG over local documents and with structured output, and I would keep the context length moderate.

Q15. What security risks exist when exposing a local LLM server to a network?

By default Ollama has no authentication. If it is opened to the network (for example with OLLAMA_HOST=0.0.0.0), anyone who can reach it can use my CPU and GPU, slow the machine down, and manage models through the API. Prompts, answers and loaded documents could be read, especially since the traffic is plain HTTP. Documents used by RAG can also contain prompt-injection instructions. To reduce these risks I would keep the server bound to 127.0.0.1 (as my Flask app is), use a firewall or VPN, place a reverse proxy with authentication, TLS and rate limiting in front of it, keep the software updated, treat retrieved documents as untrusted, and never run Flask in debug mode on a shared network.

5. Conclusion

This lab showed that a complete AI application can run entirely on a personal computer. Ollama makes the model available through a simple local API, and with a few lines of Python I built a chatbot with memory, a document-grounded assistant and a web interface. The RAG test confirmed that grounding the model in documents lets it admit when it does not know, instead of inventing an answer.

The benchmark showed that model size has a direct cost. On my machine gemma3 was about three times faster overall, loaded much more quickly and used about half the memory of qwen3.5. For this computer and for course-question answering, I would choose gemma3, and I would only move to the larger model if a task clearly needed better quality and the hardware allowed it.